

Project Stem: Building a Self-Differentiating AI Agent by Alexandru Racovita

When I first read the task description comparing AI to a biological stem cell, it really made sense to me. Usually, when we build AI agents, we act like the creators. We give them specific tools, write their prompts, and tell them exactly what to do. But a stem cell doesn't work like that. It just has a basic set of instructions and reads its environment to figure out what it needs to become. To replicate this in code, I realized I had to change how I normally write software. Instead of building an agent for a specific problem, I had to build a blank slate program whose only job is to look at a user's prompt, figure out what tools it needs, write its own Python code on the fly, and then lock itself into that new form.

I decided to build this using Python and the OpenAI API. I intentionally avoided heavy frameworks like LangChain because, being a first-year student, I really wanted to understand the core mechanics of dynamic execution myself. I created a simple loop. First, the Stem Agent receives the task. Then, it generates a JSON response containing its new name, a new system prompt, and the raw Python code for the tools it thinks it needs.

But I knew I couldn't just trust the code it generates. So, I built a Safeguard, which acts like the immune system of the application. Before the agent transforms, the generated Python code goes to this Safeguard module where I use Python's `exec` function to try and run the code in a controlled way. If it works, the tools are saved and the agent transforms. If the code throws an error, the Safeguard catches it, stops the execution, and sends the exact error message back to the LLM telling it to fix the mistake. It tries this a few times before giving up.

To actually measure if this worked, I did a direct before and after comparison. I created a local file named `sales.csv` and deliberately made it very messy. I left some rows blank and typed random text in columns where only numbers should be. The task was to read the file, clean the bad data, and calculate the total revenue.

Before the Stem Agent, I gave the exact same prompt to a raw, standard LLM. The success rate was zero percent. Because it couldn't access my local file system, it just hallucinated a fake total or gave me a generic Python script and told me to run it myself. It required total human intervention to actually get the result.

After using the Stem Agent environment, the autonomous success rate was one hundred percent. Without me touching the keyboard again, it dynamically created a RevenueCleaner agent, read the local file, cleaned the broken rows, and successfully calculated the exact revenue of 2250 dollars. It turned a manual multi-step process into a single autonomous action.

Honestly, the part I enjoyed the most was when things failed, because the code broke constantly during development. It taught me a lot about how unpredictable LLMs can be. First of all, my Safeguard kept crashing at the very beginning because the AI was trying to be helpful and wrapped its Python code in markdown formatting. My code couldn't read markdown, so I had to build a scrubber that cleans the text before trying to run it.

Then I ran into a really weird bug with variable scoping that took me a while to figure out. The AI would correctly write the import pandas statement, and the Safeguard would say it was a success. But when the final agent tried to actually run the function on my CSV file, it crashed saying pandas was not defined. I realized that the exec function was keeping the imports trapped in a local space, and once the function was called later, it had forgotten the import. I had to create a shared dictionary to act as a global environment so the function and the imports could live together permanently.

Another funny thing that happened was the AI trying to be too smart. Instead of generating a simple Python function to clean the data, it wrote an entire Object-Oriented Class. My script was just looking for a simple function to call, so it crashed when trying to execute a class directly. I had to go back to my original system prompt and be much more strict, explicitly telling the AI not to use classes and to only write standalone functions.

These failures really proved to me why the Safeguard concept from the prompt is so important. Without a feedback loop that catches errors and forces the AI to rebuild itself, a self-evolving agent would just crash on the first try.

Looking back, the biggest limitation of my current prototype is security. Using the exec function to run AI-generated code directly on my laptop is fine for a school project or a prototype, but it is pretty risky in the real world. If I had more time, I would completely isolate the Safeguard

environment. Instead of running the code locally, I would make the Stem Agent spin up a lightweight Docker container. The tools would be tested and executed inside that sandbox, so if the code does something bad or crashes, the container is simply destroyed without affecting my actual computer.